

Daniel Hernandez

CSCI-611

Bo Shen

Summer 2025

Assignment 3

Below is a table of changes I made during the tuning process. Each row represents a different iteration with specific parameter modifications. The results and actual changes are described below along with their corresponding output image.

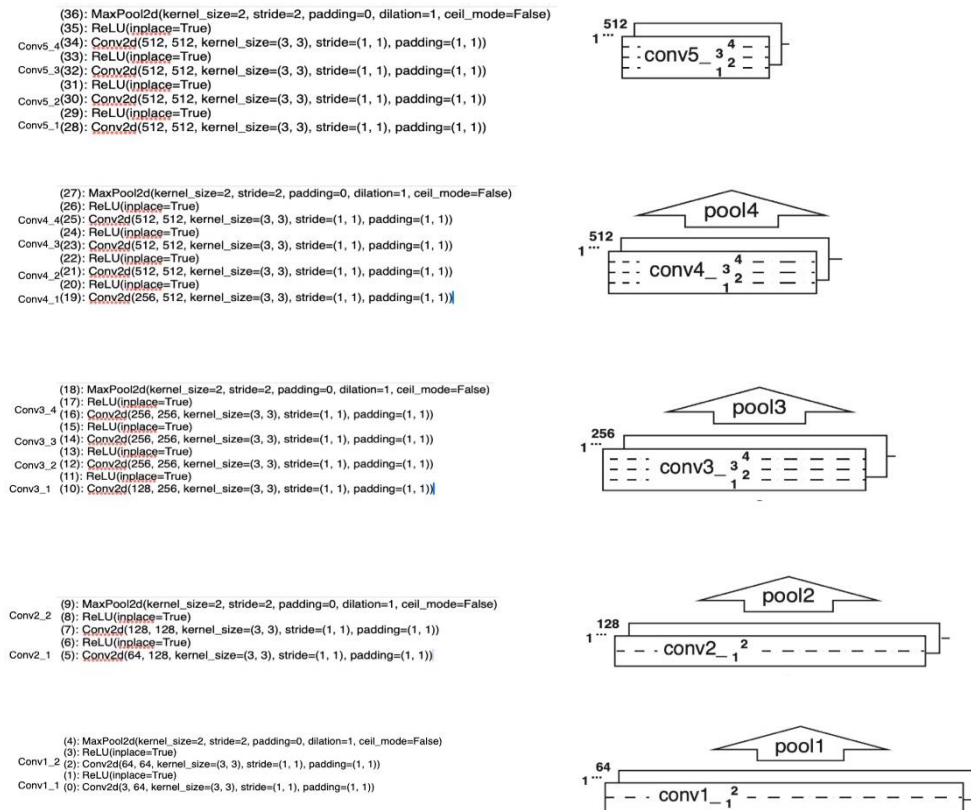
	content weight	style weight	style_weights					LR	steps
			conv1_1	conv2_1	conv3_1	conv4_1	conv5_1		
0	1	1.00E+06	1	0.8	0.5	0.3	0.1	0.003	500
1	1	1.00E+05	1	0.8	0.5	0.3	0.1	0.003	100
2	1	1.00E+04	1	0.8	0.5	0.3	0.1	0.003	100
3	1	1.00E+02	1	0.8	0.5	0.3	0.1	0.003	50
4	1	1.00E+00	1	0.8	0.5	0.3	0.1	0.003	50
5	1	1.00E+02	1	0.8	0.5	0.3	0.1	0.003	50
6	1	1.00E+05	1	0.8	0.5	0.3	0.1	0.003	50
7	1	1.00E+07	1	0.8	0.5	0.3	0.1	0.003	50
8	1	1.00E+07	1	0.8	0.5	0.3	0.1	0.003	100
9	1	1.00E+07	1	0.8	0.5	0.3	0.1	0.003	500
10	1	1.00E+07	1	0.8	0.5	0.1	0	0.003	50
11	1	1.00E+07	1.5	1	0	0	0	0.003	50
12	1	1.00E+07	1.5	0	0	0	0	0.003	50
13	1	1.00E+07	0	0	0	0	1	0.003	50
14	1	1.00E+07	0	0.7	1.5	0.7	0	0.003	50
15	1	1.00E+07	1	1	1	1	1	0.003	50
16	1	1.00E+07	1	1	1	1	1	0.001	50
17	1	1.00E+07	1	1	1	1	1	0.005	50
18	1	1.00E+07	1	1	1	1	1	0.01	50
19	1	1.00E+07	1	1	1	1	1	0.01	1000

TODO 1: The image I used for content is an image of the golden gate bridge and the image I used for the style is a famous expressionism painting by Edvard Munch in the late 19th century.



TODO 2: The content and style layers were found in the results section of the paper: “The images shown in Fig 3 were synthesised by matching the content representation on layer ‘conv4 2’ and the style representation on layers ‘conv1 1’, ‘conv2 1’, ‘conv3 1’, ‘conv4 1’ and ‘conv5 1’”

The figure below shows the corresponding image layers.



TODO 3: Using the `.size()` method, I extracted the dimensions of the tensor. Then, I reshaped it to a 2D matrix using the `.shape()` method. To compute the Gram matrix, I used `torch.mm()` along with `.t` (for transposition).

```
## get the batch_size, depth, height, and width of the Tensor
batch_size, d, h, w = tensor.size()

## reshape it, so we're multiplying the features for each channel
tensor = tensor.reshape(d, w*h)
## calculate the gram matrix
gram = torch.mm(tensor, torch.t(tensor))
```

TODO 4: Using `get_features()`, I passed the target image through the network and extracted the feature maps. Then, I calculated the content loss using the provided `content_loss` calculation.

```
## TODO: get the features from your target image
## Then calculate the content loss
target_features = get_features(target, vgg)
content_loss = torch.mean((target_features['conv4_2'] - content_features['conv4_2'])**2)
```

TODO 5: I computed the Gram matrix of the target feature map by passing target feature as an argument in the `gram_matrix` function. Here, we are extracting meaningful relationships between different feature channels.

```
## TODO: Calculate the target gram matrix
target_gram = gram_matrix(target_feature)
```

TODO 6: Earlier, I computed the Gram matrix for the style image. Here I am retrieving it. The goal of the target image is to match this representation as closely as possible.

```
## TODO: get the "style" style representation
style_gram = style_grams[layer]
```

TODO 7: Here, I am computing the loss for style at a single layer. This is achieved by taking the mean squared difference between the target Gram matrix and the style Gram matrix.

```
## TODO: Calculate the style loss for one layer, weighted appropriately
layer_style_loss = style_weights[layer] * torch.mean((target_gram - style_gram) ** 2)
```

TODO 8: Lastly, I combined the content loss and the style loss to get the total loss. The content and style losses were each weighted to control how much they affect the final image.

```
## TODO: calculate the *total* loss
total_loss = content_weight * content_loss + style_weight * style_loss
```

1. Structure gets even more blurry towards the end of the bridge. Decreasing style_weight again.



2. Almost no improvement. Decreasing style_weight to 100



3. Again, almost no difference. The style is dominating. Last resort is to decrease it to 1



4. Finally saw results. The reason for this being that I didn't run the "Putting it all together" cell after each try. That cell recreates the target image. So, when each attempt I made started from the previous attempt. Now it starts with a fresh target image. Start again and increase the style_weight to 1e2



5. Still very minimal changes. Will increase it back to $1e5$



6. Not getting much change. Increasing style_weight to $1e7$



7. No change. Going to increase the number of steps



8. Slightly better style. Increasing the step count to 500



9. Substantially good improvement, but entire process took 15 minutes. Will reduce steps back down to 50 and focus on tuning the style layer weights. Afterwards I will increase the steps back up possibly to 1000 at the very end. It may take 30 minutes



10. Reducing layers 4 and 5 did not have much effect. Will make later layers almost none and make the earlier layers very high



11. Changes are imperceptible. Going to test to see if zeroing everything out and keeping only the first layer active will produce any results



12. The image was blurred a bit, but not significant. Testing in the opposite direction, keeping only the last layer active.



13. Similar imperceptible changes. Next I will emphasize the middle layers



14. Noticeable improvement. As a test, I will set all layers to 1



15. That actually has just as good improvement as before. Now going to tune learning rate



16. Decreased from before. Going to try a higher rate



17. Significant improvement. Increasing to 0.01



18. Great improvements. Now going to increase the number of steps to 1000



19. Final image. Tower on the far right is still preserved, unlike the initial iterations.

